



POLITECNICO
DI TORINO

Dipartimento
di Automatica e Informatica

Grafo regolare e cammino semplice a peso massimo

ALGORITMI E PROGRAMMAZIONE IN PRATICA

4.3.1: TEMA DI ESAME DEL 24/2/2014

CON CONSIGLI PRATICI PER L'ESAME



Premessa: come leggere il testo di esame

Lettura e comprensione di un testo di esame (come della descrizione di un qualsiasi problema) richiedono (prima di passare a risolvere il problema):

- Tempo
- Capacità di analisi e comprensione del testo
- Individuazione (o formulazione) di un modello astratto del problema

Il tempo complessivo all'esame è di 1h30 CIRCA (leggermente superiore se online)

- Opportuno riuscire a leggere, capire e formulare un modello in 15 minuti circa (approssimativo: può essere meno o più, ma non dovrebbe essere, ad esempio, 45 minuti)
- SI CONSIGLIA DI SCEGLIRE RAPIDAMENTE parte da 12 o 18 punti

Come affrontare il testo di esame da 18 punti

Il testo comprende:

- Descrizione del problema:
 - contesto, obiettivo (serve a capire di cosa si tratta)
- Descrizione dei dati (formati del/dei file in input)
- Domande (di solito in forma esplicita, come elenco)
 - Una domanda relativa alla lettura del file e/o all'acquisizione dei dati 
 - Una domanda relativa a un sotto-problema o problema semplificato
 - Es. un sotto-problema di verifica utile per la/le domanda/e successiva/e 
 - Uno o più problemi da risolvere:
 - ricerca o ottimizzazione 
 - Gestione di strutture dati
- Un esempio (figure, testo, numeri)

Consigli pratici

Dedicare il tempo necessario a capire, ma NON ESAGERARE:

- Se c'è un esempio, guardarlo in parallelo alla lettura del testo
- Iniziare a capire le parti più semplici, poi approfondire
- Lettura a più iterazioni
- Allenarsi su esami passati, per adottare una strategia (esistono strategie, anche documentate, per una lettura efficace di un testo)

Il tempo è limitato:

- Scegliere come distribuire tempo ed energie per massimizzare il risultato
 - Se non si pensa di riuscire a fare tutto, meglio “prima parti (percepite come) più facili”? Oppure più difficili? (entrambe le strategie hanno senso).
- Valore:
 - Parti che VALGONO di più (di solito le domande più difficili, quali un problema di ottimizzazione, che può VALERE anche metà o più del punteggio complessivo).
 - Parti che valgono poco: es. Il main (dipende dai casi), o le funzioni di input e inizializzazione della struttura dati (benchè possano valere alcuni punti, raramente – SCELTA VOLUTA – sono sufficienti a ottenere la soglia minima di ammissione all'orale)



Uso degli ADT noti (le funzioni standard)

Criterio generale:



- Non si vuole valutare in modo DIRETTO (nella prova di esame scritto) lo studio/conoscenza di ADT GIA FATTI (che spesso si copierebbero, con piccole modifiche)
- Le difficoltà sono spesso in altre parti del problema (non risolvibili DIRETTAMENTE da ADT e relative funzioni già fatte)

ADT e/o funzioni “standard”:

- E' sufficiente indicarli/usarli (chiamare le funzioni) scrivendo/includendo, ad esempio, solo l'interfaccia (il .h o il suo contenuto)
 - Piccole differenze tra esame su carta o su piattaforma exam, ma la sostanza non cambia

Esame: cosa fare e cosa NO!

I test di esame suggeriscono spesso (specie la parte da 18 punti) di omettere parte del programma (queste parti andrebbero poi incluse nella relazione/verifica):

- le eventuali **funzioni "standard", di "libreria"**.

Che cosa è “standard”/”libreria” ?



- Non si può/vuole dare una definizione rigorosa
- Si possono intendere come tali le strutture dati classiche e le relative funzioni studiate (specie nella parte di “teoria”)
- Esempi:
 - ordinamento per vettori
 - funzioni su liste, ordinate e non
 - funzioni su FIFO, LIFO, code prioritario, BST, tabelle di hash, heap, grafi e altro

Esame: cosa fare e cosa NO!

Non si accettano come "standard" i modelli di funzioni ricorsive per il calcolo combinatorio

il modello in se è troppo generico e va integrato con

- la rappresentazione della soluzione,
- la gestione dei casi terminali con
 - eventuale verifica della soluzione,
 - l'eventuale pruning,
- i criteri per determinare la soluzione ottima, ecc.

Esame: cosa scrivere, cosa NO

Delle funzioni omesse (solo chiamate) è opportuno riportare

- il prototipo (se ordinamento di dati non standard, indicare anche funzione di confronto o istruzione di confronto)

Di eventuali ADT utilizzati

- il file header (il .h) o il suo contenuto
 - allegando il foglio stampato (per esame su carta)
 - oppure scrivendone direttamente il contenuto
- La definizione della struct wrapper (nel caso di un ADT per collezioni) può essere o meno inclusa in funzione di
 - programma completamente "client" dell'ADT (inutile descrivere il wrapper)
 - oppure si sviluppi una parte dell'ADT (meglio in tal caso definire la struct wrapper)

Devo/posso? Giusto/sbagliato?

La parte da 18 punti è “progettuale”

Il concetto di giusto/sbagliato NON è rigoroso:

- Ci sono (talvolta) parti soggette a interpretazione:
 - La libera interpretazione non modifica la difficoltà del problema (es. Come stampare il risultato)
 - Ci potrebbero (non voluti) essere piccoli errori o incongruenze
 - Quasi sempre indicate/spiegate in sede di esame, oppure risolvibili indicando in modo esplicito l'interpretazione adottata
- Ci sono errori dal peso MINIMO o TRASCURABILE
 - li fanno tutti, si correggono rapidamente (compilazione e/o debug)
- Si vuole valutare la CAPACITA' di RISOLVERE PROBLEMI IN C, usando le STRATEGIE INSEGNATE

Devo/posso? Giusto/sbagliato?

La parte da 18 punti è “progettuale”

Il concetto di giusto/sbagliato NON è rigoroso:

- Ci sono (talvolta) punti soggette a interpretazione:
 - La libera interpretazione non modifica la difficoltà del problema (es. Come stampare)
 - Ci potrebbero essere piccoli errori o incongruenze

■ La risposta più frequente a “devo/posso?” sarebbe:

- Fai le scelte che ritieni più opportune, valuteremo.
- Vogliamo valutare sia la capacità di scegliere che di realizzare (non solo la seconda)

Esempio – Esame 24/2/2014: Problema

- Acquisire un grafo in una opportuna struttura dati
- verificare se il grafo è regolare o meno
- iterare (finche non si legge “fine”) su:
 - leggere un identificatore (nome vertice)
 - determinare il cammino semplice di peso (somma pesi archi) massimo con sorgente nel vertice indicato
 - Per il cammino trovato visualizzare gli archi e il peso totale.

Il grafo

Grafo orientato e pesato

File: Rappresentazione come elenco di archi.

- La prima riga del file riporta il numero di vertici V
- Vertici: V righe con nome di vertice (stringa alfanumerica di max 20 caratteri).
- Archi: Seguono righe con archi pesati (uno per riga), nel formato
 - idV1 idV2 val :
il vertice con identificatore idV1 è connesso con un arco orientato di peso val (valore intero e positivo) al vertice idV2 .

Soluzione:

- ADT grafo (si tratta di un lavoro standard o quasi) a cui si aggiungono, alle funzioni standard, le funzioni per
 - verifica di regolarità (GRAPHregular): solo risultato binario (vero/falso)
 - cammini semplici massimi (GRAPHlongestPath): con stampa risultato
- vertici del grafo con nome (stringa di caratteri). Si può usare tabella di simboli (ST)

Strutture dati: tabella di simboli

- Usata per due scopi:
 - da stringa a indice (Stsearch)
 - da indice a stringa (STsearchByIndex) DI SOLITO NON FORNITA (seconda chiave -> un'altra tabella)

ADT di I classe realizzato mediante vettore

- Non ordinato per ricerca stringhe
 - ad accesso diretto per indici
- Tabella interna al grafo, utilizzata da funzioni di input/output (GRAPHload, GRAPHlongestPath)
- se l'ADT Graph fornisce una funzione GRAPHsearchByKey (da nome a indice) il MAIN non ha bisogno di accesso diretto alla tabella di simboli

Verifica di regolarità

Un grafo è regolare se tutti i vertici hanno lo stesso grado in ingresso e uscita, e questo grado è lo stesso per tutti i vertici.

Per verificare le due condizioni occorre:

- Calcolare, per ogni vertice v , i due gradi $in_d[v]$ e $out_d[v]$ (mentre si costruisce il grafo, o nella funzione di verifica)
- Verificare
 - Per ogni vertice v : $in_d[v] == out_d[v]$
 - Per ogni coppia di vertici (v,w) : $in_d[v] == in_d[w]$ (così facendo si avrebbe un algoritmo $O(V^2)$, ma notando che è sufficiente confrontare tutti i vertici con uno qualunque, scelto in modo arbitrario, si può fare verifica $O(V)$ con $in_d[v] == in_d[0]$)

Cammini ottimi

Problema di ottimizzazione basato sui cammini semplici:

- cammino semplice avente massima somma dei pesi

L'enumerazione dei cammini semplici può essere vista come un algoritmo ricorsivo che enumeri

- tutte le permutazioni dei vertici,
- oppure le disposizioni senza ripetizioni (con k ignoto)

Con il vincolo che due vertici successivi devono essere adiacenti nel grafo (connessi da un arco).

Usando pruning, basato su matrice o liste di adiacenza, è possibile evitare (per costruzione) i cammini non validi.

L'algoritmo finale è simile alla visita in profondità ricorsiva (ma con backtrack, che permette di visitare, in un nuovo cammino, un vertice già visitato in un cammino precedente).